

Реализация PINN с использованием библиотеки Numpy и Numba (CUDA)

Свиридов Никита, студент 4 курса 2024

1 Введение

PINN - physical informed neutron net. Эта нейросеть предназначена для решения дифференциальных уравнений.

Данную задачу можно считать задачей без учителя, потому что для обучения этой нейросети не нужна разметка. Для расчета лосса используются само дифференциальное уравнение, а также граничные и начальные условия. Принцип обучения нейросети будет описан ниже, однако сейчас можно сказать, что нейросеть обучается таким образом, чтобы при подстановке функции, которая лежит в основе нейросети в наше ДУ, ГУ и НУ отклонения от истинных значений были минимальны.

2 Дифференциальное уравнение

В качестве уравнения я взял уравнение теплопроводности, которое представляет из себя уравнение в частных производных до 2 порядка.

$$\left\{ \begin{array}{l} \frac{\partial u}{\partial t} = \Delta u + xt^2 \sin y, \quad 0 < x < 1, \quad 0 < y < \frac{\pi}{2}, \quad t > 0 \\ \frac{\partial u}{\partial x} \Big|_{x=0} = \frac{\partial u}{\partial x} \Big|_{x=1} = 0 \\ u \Big|_{y=0} = \frac{\partial u}{\partial y} \Big|_{y=\pi/2} = 0 \\ u \Big|_{t=0} = 0 \end{array} \right.$$

3 Архитектура нейросети

В качестве архитектуры я выбрал многослойный персептрон, состоящий из 4 слоев, между которыми в качестве функции активации я взял сигмоиду. Моя архитектура:

$$\sigma(\sigma(\sigma(X^{n \times 3} A_1^{3 \times 32} + B_1^{1 \times 32}) A_2^{32 \times 32} + B_2^{1 \times 32}) A_3^{32 \times 32} + B_3^{1 \times 32}) A_4^{32 \times 1} + B_4^{1 \times 1})$$

$A_1^{m \times k}$, $B_1^{1 \times k}$ - матрица параметров и свободный член соответственно. Сложение выполняется вдоль оси столбцов к каждой строчке из A .

$$\sigma = \frac{1}{1 + e^{-x}} - \text{функция активации сигмоида}$$

Для инициализации весов матриц в линейных слоях я использовал метод Xavier, суть которого в заполнении элементов матрицы случайным шумом из нормального распределения $Norm(0, \frac{1}{\sqrt{\dim_{out}}})$.

4 Реализация нейросети

Можно выделить 2 этапа построения нейросети. Первый это написание самого алгоритма обучения. Второй это перенос вычислений на CUDA.

4.1 Алгоритм

Нейросеть можно представить как большую сложную параметризованную функцию. Процесс обучения параметров этой нейросети как любой другой основывается на методе градиентного спуска, а именно разрабатывается некоторая функция потерь и в процессе обучения вычисляется ошибка по этой функции. На основе этой ошибки вычисляются градиенты параметров нашей модели как сложной функции. Так как градиент представляет собой направление роста некоторой функции то исходя из цели минимизировать нашу ошибку мы изменяем наши параметры нейросети в сторону антиградиента.

Я предпочел не высчитывать формулы градиентов для слоев модели вручную, а написать общий вид нейросети и обучать уже ее. Под общим видом я подразумеваю систему автоматического дифференцирования. Система автоматического дифференцирования - это по факту класс-обертка (class tensor) над обычным тензором, который обладает свойством запоминать градиент по арифметической операции на каждом шаге вычислений. Благодаря этому свойству всегда можно рассчитать градиент любого элемента в графе вычислений путем метода обратного распространения ошибки.

4.2 Параллельные вычисления

Для того чтобы нейросеть работала за разумное время было принято решение о переносе вычислений на CUDA. Однако необходимо было решить проблему аллокации данных, ведь процесс обучения нейросети - это динамический процесс (на каждом шаге вызываются сотни конструкторов и деструкторов нашего класса). Переносить матрицы на видеокарту в время обучения модели, будет неправильным. Такой подход лишает нас всех преимуществ параллельных вычислений, потому что это занимает очень много времени. Было принято решение аллоцировать заранее большое количество памяти, которое хватит для одной эпохи обучения модели и работать в нем. Однако как я сказал только для одной эпохи, потому что в процессе обучения модели тысячи эпох. Суммарное количество памяти, требуемое на весь процесс обучения вылезает за ограничение в 15Gb GPU.

С целью решения поставленной проблемы, я пишу класс обертку на аллоцированной памятью `__memory_manager__`, на который делегирую оптимизационную задачу выделения и очистки памяти в нашем массиве. Когда требуется создать новый в нашем аллоцированном массиве выбирается свободный участок и помечается как занятый. Когда вызывается деструктор тензора данный участок памяти становится снова доступным для выделения памяти. Для выбора участка памяти при создании нового тензора используется жадный алго-

ритм для минимизации фрагментации памяти. Идея заключается в поддержке отрезков свободной памяти в отсортированном виде в списке. Поиск наиболее оптимального отрезка для выделения в нем памяти осуществляется за логарифмическое время с помощью бинарного поиска.

Функции для распараллеливания можно разбить на 3 категории:

- Арифметические операции
- Заполнение памяти
- Reduce операции

Арифметические операции. Для полноценной работы нейронной сети были реализованы следующие операторы: матричное произведение; поэлементное сложение, умножение, деление, возведение в степень; поэлементное вычисление функций $\ln$, $\cos$, $\sin$. В реализации матричного умножения используется блочный алгоритм. Все вышеупомянутые бинарные операторы предопределены также для случая, когда в качестве второго элемента число, а не класс `tensor`.

Заполнение памяти. Также параллелизация была применена алгоритмам заполнения памяти. Были реализованы алгоритмы заполнения матрицы другой матрицей, числом; алгоритмы взятия части матрицы по срезу и заполнения матрицы по срезу.

Суммирование элементов. Эта операция попадает под парадигму `reduce`, параллельная реализация которой представляет особую сложность. Реализованы суммирование всех элементов, суммирование вдоль оси столбцов и суммирование вдоль оси строчек.

*Все алгоритмы можно посмотреть в коде

5 Обучение нейросети

Как было сказано ранее наш алгоритм основан на градиентном методе. В этом разделе я привожу описание функций потерь на минимизацию которых обучается наша нейросеть.

Функция ошибок. Loss представляет собой сумму трех функций. Первая отвечает за выполнения дифференциального уравнения, вторая за граничные условия, третья за начальные.

$$Loss = \left\| \frac{\partial u_{pinn}}{\partial t} - \Delta u_{pinn} - xt^2 \sin y \right\| + \sum_{i=1}^4 \|BC_i(u_{pinn}) - BC_i(u)\| + \|Init(u_{pinn}) - Init(u)\|$$

BC_i - оператор граничных условий,

$Init$ - оператор начальных условий

Параметры обучения. Выше было сказано, что процесс обучения основан на изменении весов нашей модели в сторону антиградиента функции потерь, однако простое его вычитание не является наилучшим решением. Существуют более продвинутые градиентные методы оптимизации. Одним из таких является метод градиентного спуска Adam, который используется в моей реализации

Что касается параметров модели, в качестве learning rate наиболее оптимальным оказалось значение 0.01, количество эпох было выбрано 2000 исходя из компромисса между временем обучения и точностью решения.

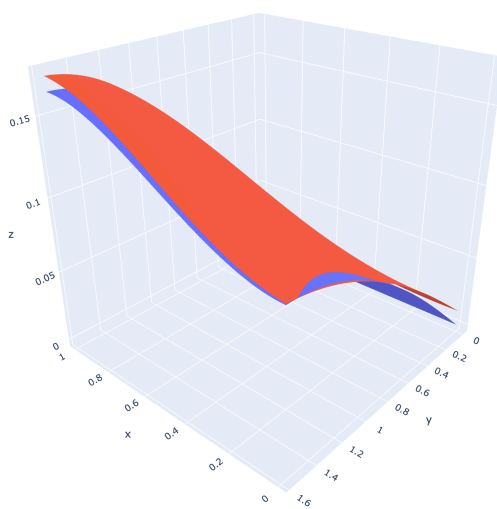
Размер батчей в лосс функциях. На каждой эпохе обучения генерируются случайным образом точки из области пространства для которого мы ищем решение. Причем в случае подсчета ошибки для ДУ точки генерируются случайно из 3 мерного пространства пространства (x, y, t), а для НУ и ГУ фиксируется одна из координат. Было принято решение считать ошибку усредненную по 200 точкам для лосса ДУ и по 50 точкам для лоссов, отвечающих за ГУ и НУ

6 Сравнение с аналитическим решением

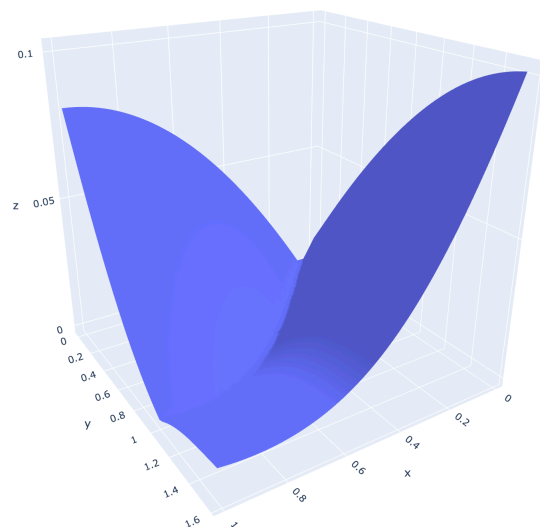
Максимальное абсолютное отклонение не превышает 0.1. Ниже приведены графики сравнения с аналитическим решением и подсчет ошибки решения для времен $t = 0.5$ с.

Говоря об актуальности решения ДУ с помощью PINN, я сошлюсь на библиотечное решение, которое я также реализовал. Выше я уже упоминал, что мое решение в 10 раз медленнее которое я построил с помощью библиотеки Pytorch. Поэтому в случае библиотечного решения количество эпох может быть существенно увеличено и точность увеличится на порядки. Это является основанием для составления конкуренции другим численным методам решения ДУ.

Сравнение с аналитическим решением



Ошибка решения



Синим цветом обозначено аналит. решение, красным - численное

7 Время выполнения программы

Было проведено сравнение алгоритмов автоматического дифференцирования с вычислениями на CUDA и при вычислениях на обычном NumPy. Использование CUDA позволяет нам ускорить вычисления в 8 раз. Такой небольшой прирост обусловлен небольшими размерами матриц использующих в нашей архитектуры. На протяжении всего графа вычислений не встречается матрица с размером по любой оси больше 200.